운영체제

4주차 – CPU 스케줄링

학습 목표

- 스케줄링을 이해한다.
- 우선순위를 이해한다.
- CPU 스케줄링 알고리즘을 이해한다.

스케줄링

요리사 모형에서의 스케줄링

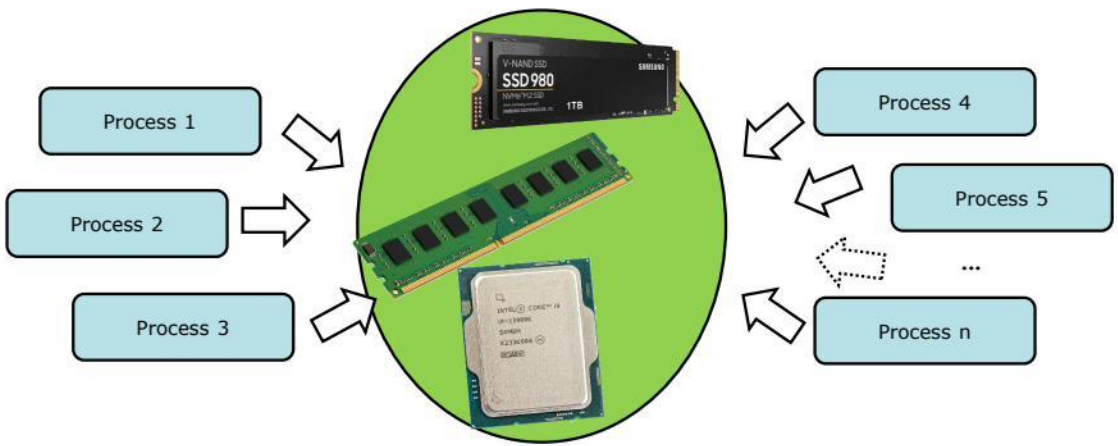
- **CPU 스케줄러**
  - 운영체제에서 **레스토랑 관리자**의 역할을 담당
  - 여러 프로세스 상황을 고려하여 CPU와 시스템 자원의 배정을 결정

그림 4-1 레스토랑 관리자의 역할

그림 4-2 조리 순서 변경

스케줄링 개요 (1/3)

- **필요성**
  - 작업(프로세스)은 주어진 일을 수행하기 위해 다양한 자원(resource)을 사용
    - 자원 예: CPU, memory, storage, network, file, I/O etc.
    - 자원의 특징 : 유한함
  - CPU는 작업(프로세스) 단위로 처리
  - 작업의 개수가 많아지면 자원들을 선점하기 위한 경쟁이 필요
  - 작업 간의 효율적인 자원의 할당 및 공유를 위한 관리자가 필요 ← **스케줄러(Scheduler)**



스케줄링 개요 (2/3)

- **개념**
  - 프로세스들에 대하여 CPU에 할당 기회를 결정하고 수행하는 과정
  - 프로세스가 작업을 처리하기 위해 CPU 할당을 위한 일정을 처리

- **스케줄링 정책**
  - **정해진 시간(time slice, time slot)에 프로세서를 점유할 프로세스를 선택**
  - 고려사항
    - 최대의 처리량 → 프로세서 활용도 극대화
    - 최소 지연시간
    - 무기한 연기(대기)를 방지(공평성)
    - 요청 데드라인 전에 완료

스케줄링 개요 (3/3)

- **스케줄링의 목표**

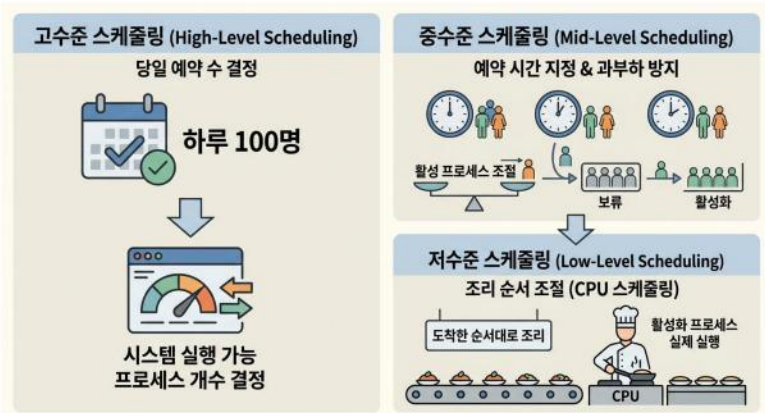
공평성	• 모든 프로세스가 자원을 공평하게 배정받아야 함 • 자원 배정 과정에서 특정 프로세스가 배제되어서는 안 됨
효율성	• 시스템 자원이 유휴 시간 없이 사용되도록 스케줄링 • 유휴 자원을 사용하려는 프로세스에는 우선권을 주어야 함 • 우선순위를 사용하여 중요 프로세스가 먼저 작동하도록 배정
안정성	• 시스템 자원을 점유하거나 파괴하려는 프로세스로부터 자원을 보호 • 프로세스가 증가해도 시스템이 안정적으로 작동
확장성	• 시스템 자원이 늘어나는 경우 이 혜택이 시스템에 반영되게 해야 함
반응 시간 보장	• 사용자가 예측하는 시간 내에 시스템은 적절한 반응해야 함
무한 연기 방지	• 특정 프로세스의 작업이 무한히 연기되어서는 안 됨

(음식전 사례를 통한) 단계별 스케줄링

- **고수준(상위단계) 스케줄링**
  - 당일 예약 수 조절 (예) 하루에 100명의 손님만!!
  - 시스템에서 동시에 실행할 수 있는 전체 프로세스의 개수 결정

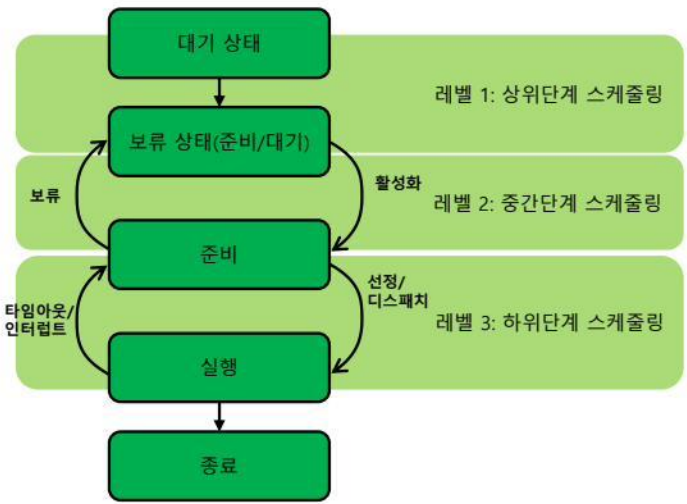
- **중수준(중간단계) 스케줄링**
  - 100명이 한꺼번에 몰리는 것을 방지하기 위해 예약 시간을 지정
  - 활성 프로세스를 조ㅓㄹ해 시스템 과부하를 방지
    - 특정 프로세스를 보류하거나 다시 활성화하여 부하 조절

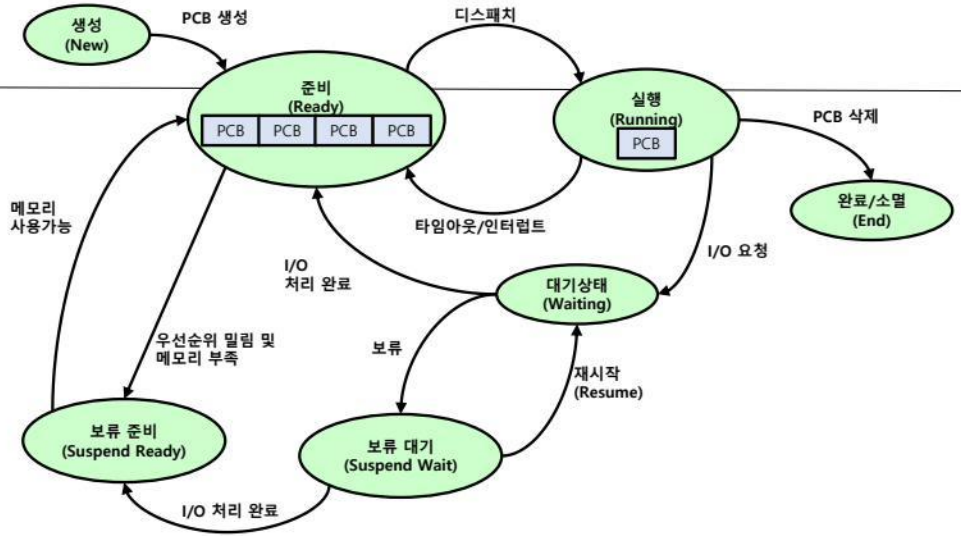
- **저수준(하위단계) 스케줄링**
  - 조리 순서 조절, 도착한 순서대로 조리
  - 활성화 프로세스가 실제로 실행하는 단계
  - CPU 스케줄링



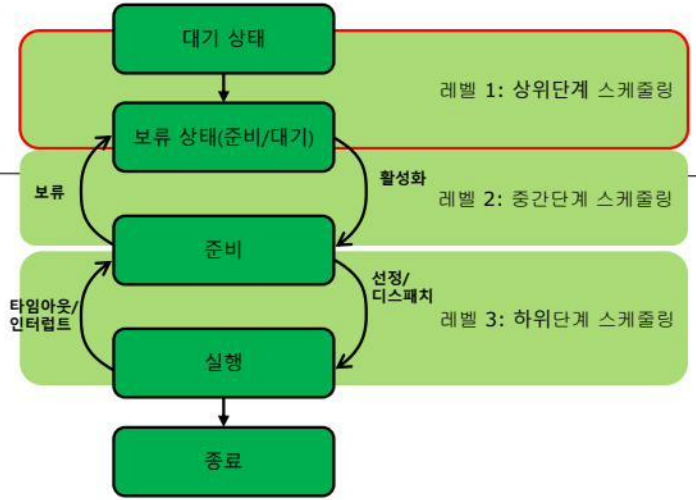
스케줄링 단계 (1/5)

- **단계**
  - 상위단계 스케줄링
  - 중간단계 스케줄링
  - 하위단계 스케줄링

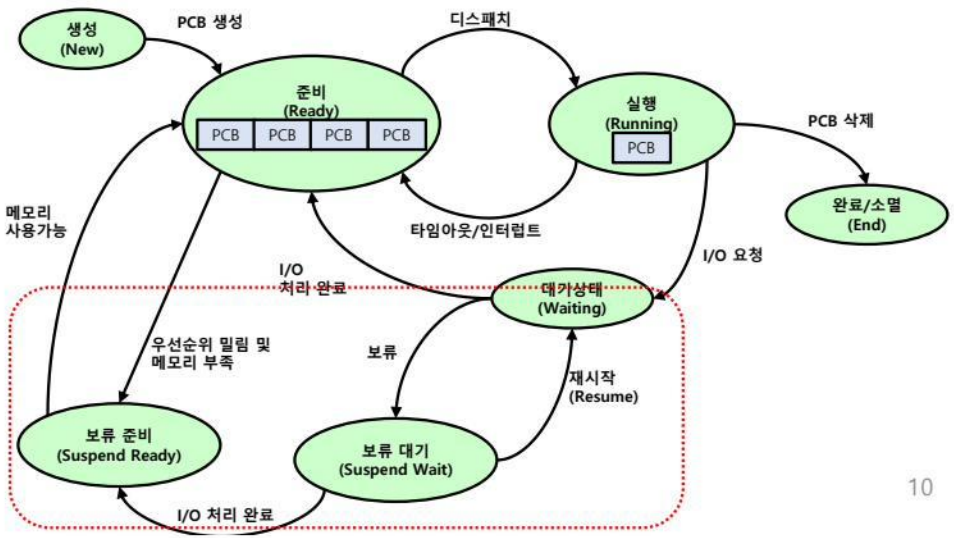




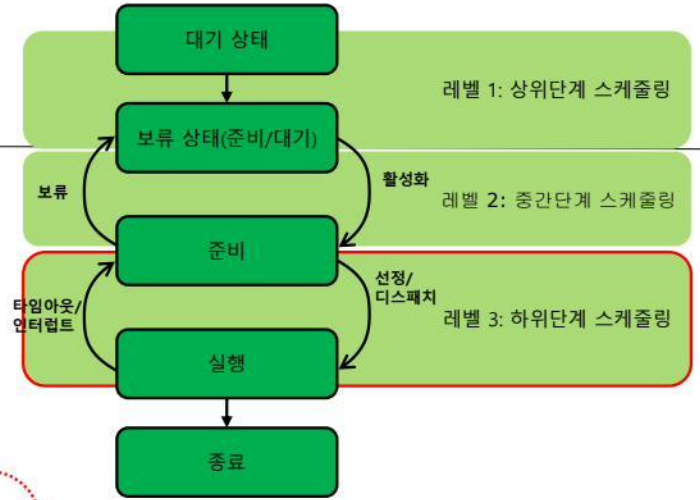
스케줄링 단계 (2/5)



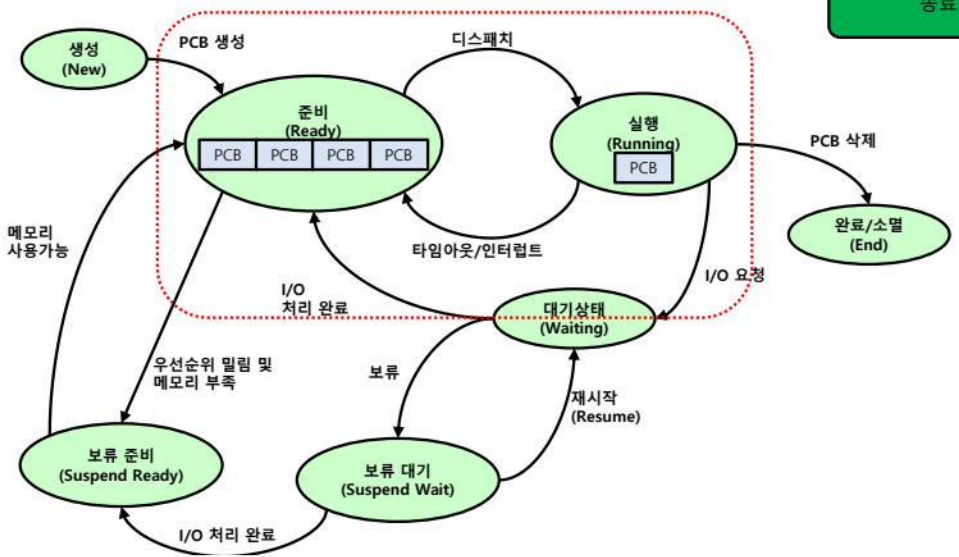
- **상위단계 스케줄링(High Level Scheduling)**
  - = 장기/작업/승인 스케줄링 (long-term/**job**/admission **scheduling**)
  - **시스템에 부담이 되지 않도록 전체 작업 수를 조절**
  - 어떤 작업을 시스템이 받아들일지 또는 거부할지를 결정
  - 실행 가능한 프로세스의 총 개수를 정의



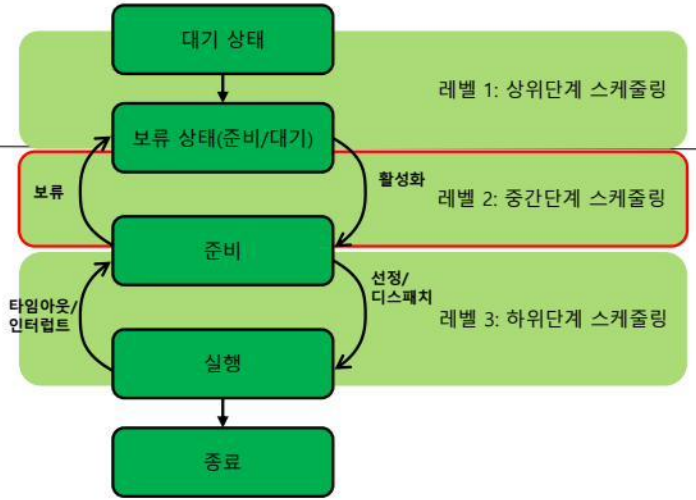
스케줄링 단계 (3/5)



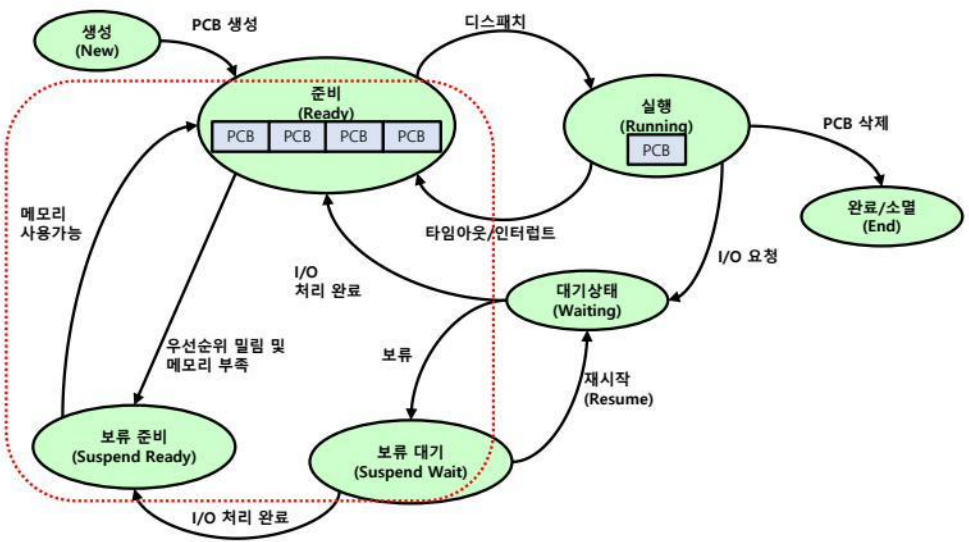
- **하위단계 스케줄링(Low Level Scheduling)**
  - = 단기 스케줄링(short-term scheduling, **CPU scheduling**)
  - 짧은 시간에 처리
  - 프로세스를 **CPU에 할당**
  - 프로세스를 **대기상태로 전환**



스케줄링 단계 (4/5)



- **중간단계 스케줄링(Middle Level Scheduling)**
  - 보류(suspend) - 활성화(active) 상태 전환
  - **활성화된 프로세스 조절** → 과부하를 해소
  - 저수준 스케줄링의 부담을 완충하는 역할



스케줄링 단계 (5/5)

- **정리**

고수준 스케줄링	전체 시스템의 부하를 고려하여 작업을 시작할지 말지 결정
중간 수준 스케일링	시스템에 과부하가 걸려서 전체 프로세스 수를 조절해야 한다면 이미 활성화된 프로세스 중 일부를 보류 상태로 보냄
저수준 스케줄링	실제 작업 수행

선점형/비선점형 스케줄링 (1/3)

- **선점형 스케줄링(Preemptive Scheduling)**
  - 운영체제가 필요하다고 판단하면 실행 상태에 있는 프로세스의 작업 중단
  - 다른 프로세스가 프로세서를 쟁취할 수 있음
    → 새로운 작업을 시작할 수 있는 방식
  - 하나의 프로세스가 CPU를 독점할 수 없음
    → 빠른 응답이 가능
  - 대화형 시스템이나 시분할 시스템에 적합
  - 대부분의 저수준 스케줄러가 이 방식을 사용

선점형/비선점형 스케줄링 (2/3)

- **비선점형 스케줄링(Non-Preemptive Scheduling)**
  - 한 프로세스가 CPU를 사용하면 종료 또는 자발적으로 나올 때까지 다른 프로세스는 대기
    - 프로세스가 스스로 양보
  - 과거의 일괄 작업 시스템에서 사용하던 방식
  - 장점
    - 선점형 스케줄링보다 **스케줄러의 작업량이 적음**
    - 문맥 교환에 의한 프로세서 및 시간적인 낭비 감소
  - 단점
    - 덩치가 큰 프로세스는 CPU 사용이 길기 때문에, 타 프로세스의 대기시간을 길게 만듦
      → 전체 시스템의 처리율이 떨어짐

선점형/비선점형 스케줄링 (3/3)

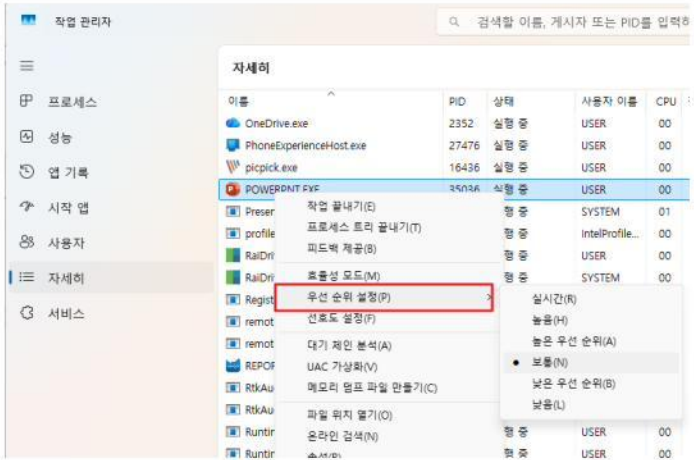
표 4-1 선점형 스케줄링과 비선점형 스케줄링의 비교

구분	선점형 스케줄링	비선점형 스케줄링
작업 방식	실행 상태에 있는 작업을 중단시키고 새로운 작업을 실행할 수 있다.	실행 상태에 있는 작업이 완료될 때까지 다른 작업이 불가능하다.
장점	프로세스가 CPU를 독점할 수 없어 대화형이나 시분할 시스템에 적합하다.	CPU 스케줄러의 작업량이 적고 문맥 교환의 오버헤드가 적다.
단점	문맥 교환의 오버헤드가 많다.	기다리는 프로세스가 많아 처리율이 떨어진다.
사용	시분할 방식 스케줄링에 사용된다.	일괄 작업 방식 스케줄링에 사용된다.
중요도	높다.	낮다.

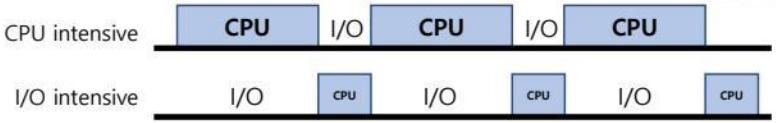
우선순위

프로세스 우선순위

- **프로세스 우선순위**
  - 커널 프로세스의 우선순위가 일반 프로세스보다 높음
  - 우선순위가 높다는 것은 더 빨리, 자주 실행되는 의미!
    - 우선순위가 높은 프로세스가 CPU를 먼저, 더 오래 차지
  - 시스템에 따라 높은 숫자가 높은 우선순위를 나타내기도 하고, 낮은 숫자가 높은 우선순위를 나타내기도 함
  - 사용자가 프로세스의 우선순위 조절 가능
    - 관리자만 우선순위 높일 수 있고, 일반 계정은 낮추는 것만 가능
    - 한 프로세스의 우선순위를 높이면, 다른 프로세스의 작업이 원활히 이루어지지 않을 수 있음



CPU 집중 프로세스와 I/O 집중 프로세스 (1/2)



- **CPU 집중 프로세스(CPU intensive)**
  - CPU burst 시간이 긴 경우
    - 프로세스가 실행되는 동안 수학 연산과 같이 CPU 사용이 많은 프로세스
  - (예) 계산을 주로 하는 비디오, 이미지 압축 프로그램, 행렬 연산하는 AI 프로그램 등

- **I/O 집중 프로세스(I/O intensive)**
  - I/O burst 시간이 긴 경우
  - 다중 프로그래밍 도입시기에 I/O burst 시간동안 CPU 유휴시간을 줄이기 위해 다른 프로세스를 동작 시킬 수 있도록 CPU 스케줄링이 도입됨

- **CPU burst**
  - 프로그램 실행과정에서 CPU가 코드를 집중적으로 실행하는 상황
- **I/O burst**
  - I/O 장치에 의해 입출력이 이루어지는 상황

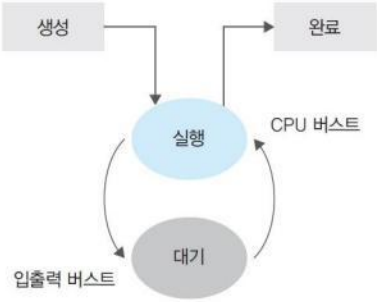
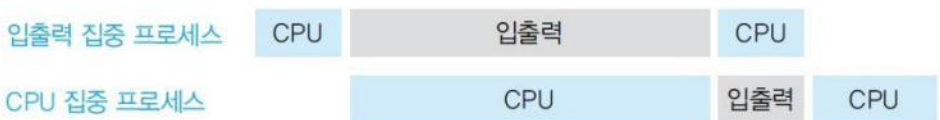


그림 4-5 CPU 버스트와 입출력 버스트

CPU 집중 프로세스와 I/O 집중 프로세스 (2/2)

- **우선 배정**
  - I/O burst 상황에서 다른 프로세스가 CPU를 사용할 수 있기 때문에 배정 순서가 성능에 영향 미침
  - 스케줄링 시 I/O 집중 프로세스의 우선순위를 CPU 집중 프로세스 보다 높이면 시스템 효율 향상
    → 사이클 훔치기(Cycle stealing)

(a) CPU 집중 프로세스 우선 배정



(b) 입출력 집중 프로세스 우선 배정

그림 4-6 CPU 집중 프로세스와 입출력 집중 프로세스의 우선 배정 결과

전면/후면 프로세스

- **전면 프로세스(Foreground Process)**
  - GUI의 경우 운영체제 화면에서 맨 앞에 놓인 활성화 상태의 프로세스
  - 현재 입력과 출력을 사용하는 프로세스
  - 사용자와 **즉각적인** 상호작용이 가능 → 상호작용 프로세스

- **후면 프로세스(Background Process)**
  - 사용자와 **상호작용이 없는** 프로세스
  - 사용자의 입력 없이 작동

    → 일괄 작업 프로세스라고도 함
  - 전면 프로세스 보다 **우선순위가 낮음**

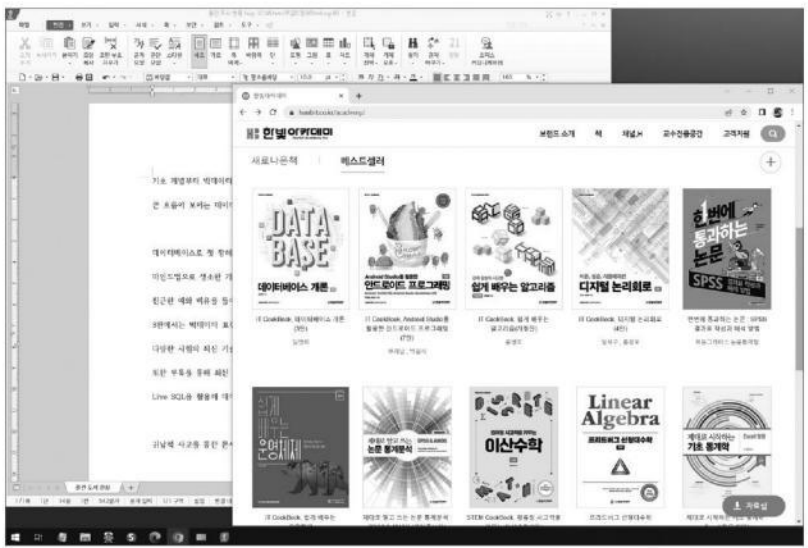


그림 4-7 전면 프로세스와 후면 프로세스의 예

정리

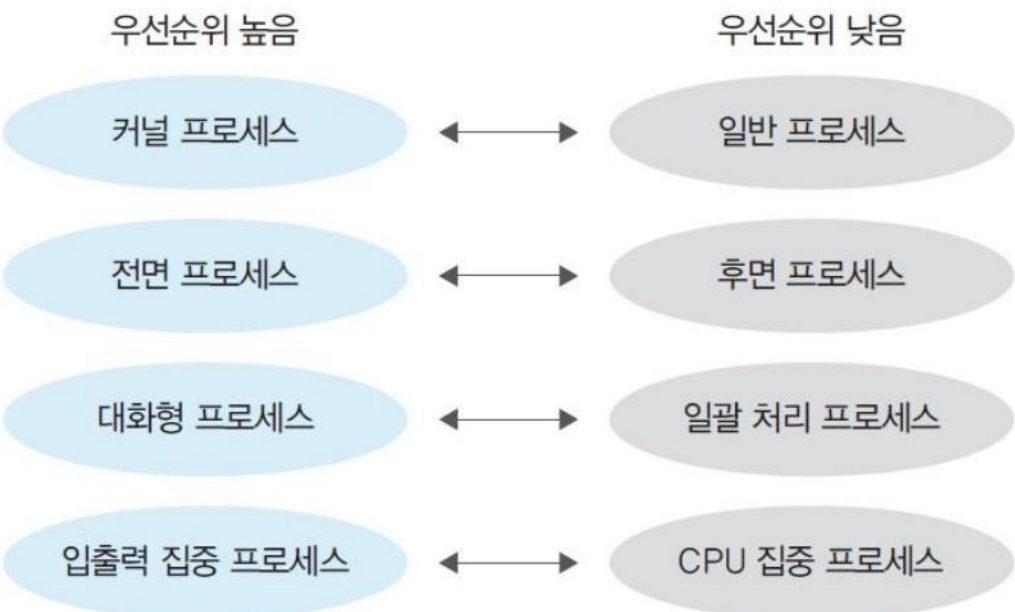


그림 4-8 CPU 스케줄링 시 고려 사항

다중 큐 (1/4)

- **준비 상태의 다중 큐(multiple queue)**
  - 프로세스는 준비 상태에 들어올 때마다 자신의 우선순위에 해당하는 큐의 마지막에 삽입
  - CPU 스케줄러는 우선순위가 가장 높은 큐의 맨 앞에 있는 프로세스에 CPU 할당 (PCB 6)
    - **한 번에 하나의 프로세스를 꺼내 CPU에 할당**

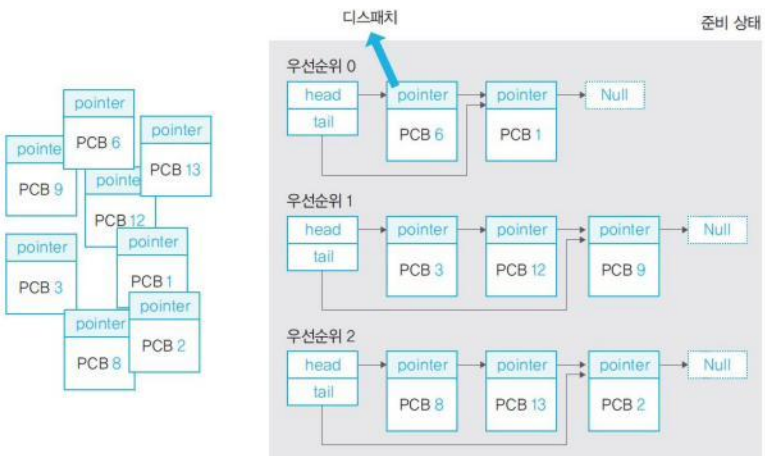


그림 4-9 준비 상태의 다중 큐

다중 큐 (2/4)

- **프로세스의 우선순위 배정 방식**
  - 고정 우선순위 방식(Static Priority)
    - 운영체제가 프로세스에 우선순위를 부여하면 프로세스가 끝날 때까지 바뀌지 않는 방식
    - 프로세스가 작업하는 동안 우선순위가 변하지 않아 구현이 쉬움
    - 빠르게 변하는 시스템 상황에 대응하기 어려워 작업 효율이 떨어짐
  - 변동 우선순위 방식(Dynamic Priority)
    - 프로세스 생성 시 부여받은 우선순위가 프로세스 작업 중간에 변하는 방식
    - 구현하기 어렵지만, 시스템의 효율성을 높임

다중 큐 (3/4)

- **대기 상태의 다중 큐**
  - 시스템의 효율을 높이기 위해 같은 입출력을 요구한 프로세스끼리 모아 놓음
  - 보통 큐의 순서대로 처리되지만, 일부는 나중에 들어온 프로세스가 먼저 처리되기도 함
  - 여러 개의 프로세스 제어 블록(PCB)를 동시에 꺼내 준비 상태로 옮김
    - 동시에 끝나는 인터럽트를 처리하기 위해 인터럽트 벡터(interrupt vector)라는 자료 구조 사용

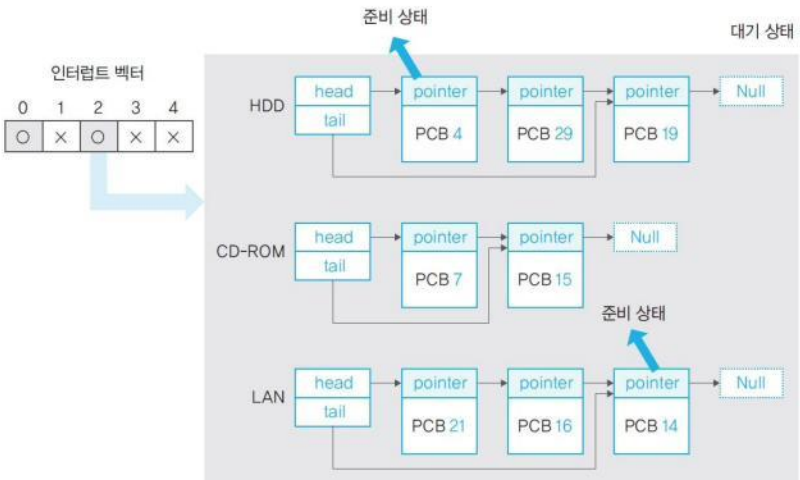


그림 4-10 대기 상태의 다중 큐

다중 큐 (4/4)

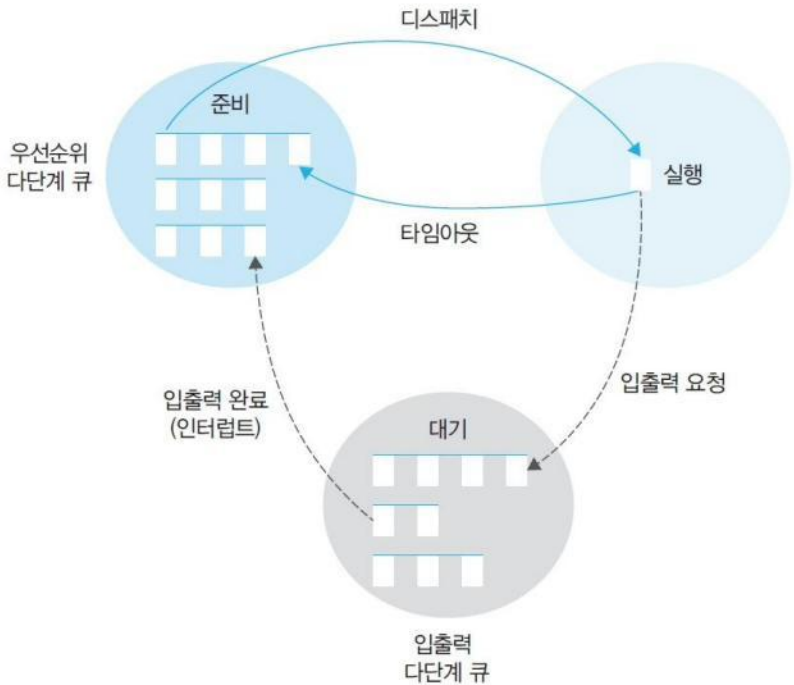


그림 4-11 프로세스 상태와 다중 큐

CPU 스케줄링

스케줄링 평가 방법 (1/3)

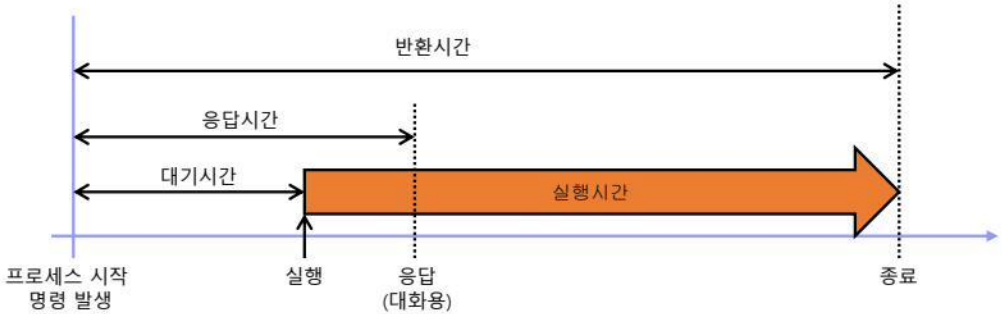
- **CPU 사용률(%)**
  - 전체 시스템의 동작 시간 중 프로세스들이 CPU를 사용한 비율을 측정
  - 높을 수록 운영체제의 성능이 좋음

$$\text{사용률}(\%) = \frac{\text{프로세스들의 처리시간}}{\text{운영체제 처리시간} + \text{프로세스들의 처리시간}} \times 100$$

- **처리량**
  - 단위 시간당 작업을 마친 프로세스의 수
  - 수치가 클수록 좋은 알고리즘

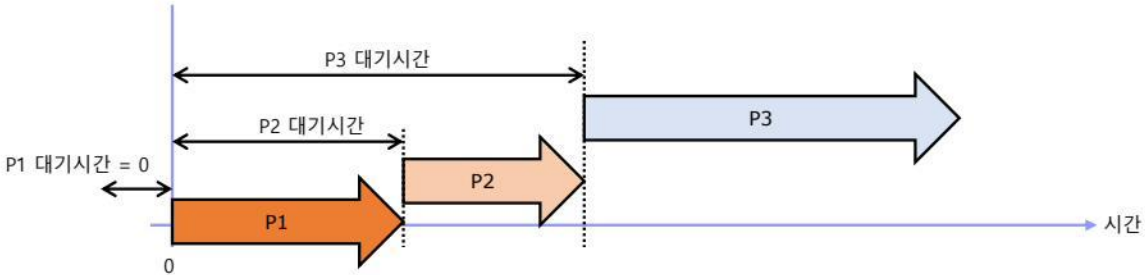
스케줄링 평가 방법 (2/3)

- **시간**
  - **대기 시간** : 프로세스가 **생성된 후 실행되기 전**까지 대기하는 시간
    - 예: 실행 파일 더블클릭 후 실제 구동시작 시간의 차이
  - **응답 시간** : 첫 작업을 시작한 후 **첫 번째 출력(반응, response, feedback)**이 나오기까지의 시간
    - 예: 실행프로그램 UI 생성된 시간
  - **실행 시간** : 프로세스 작업이 **시작된 후 종료**되기까지의 시간
    - 예: 실행프로그램의 동작시간
  - **반환 시간** : **대기 시간을 포함하여 실행이 종료**될 때까지의 시간
    - 예: 실행프로그램의 종료시간(전체 시간), 대기시간+실행시간



스케줄링 평가 방법 (3/3)

- **평균 대기 시간**

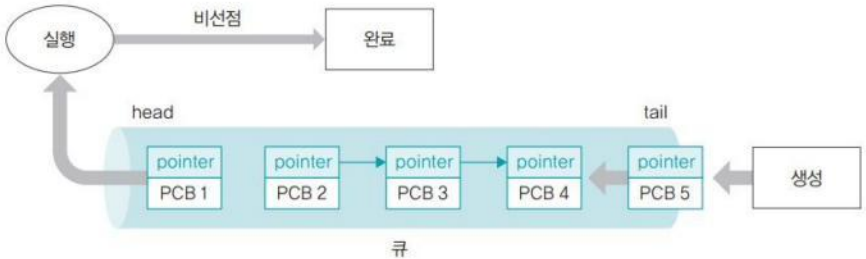


$$AVR_{(n)} = \frac{\sum_{i=1}^{n} P_{(i)}}{n}$$

- $AVR_{(n)}$: n 개에 대한 대기시간
- $P_{(i)}$: i에 대한 대기시간

FCFS 방식 (1/2)

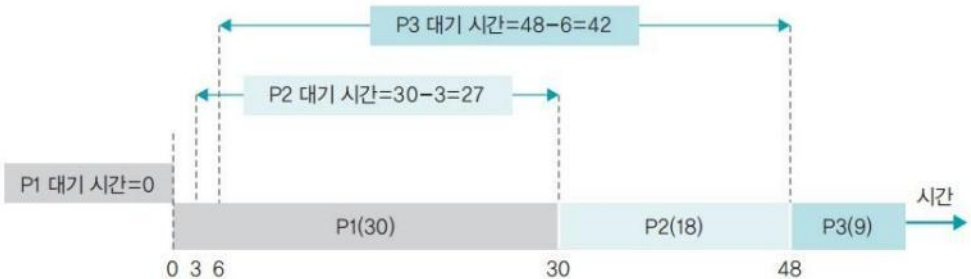
- **FCFS**
  - First Come First Served : 선입선출
  - 큐에 도착한 순서대로 CPU를 할당
  - 비선점, 우선순위 동일



- **FCFS의 평균 대기시간 계산**

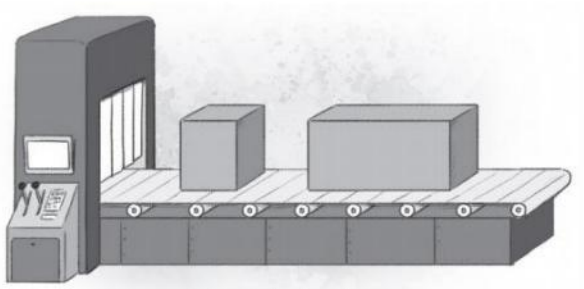
도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

$$AVR_{(3)} = \frac{W_{(p1)} + W_{(p2)} + W_{(p3)}}{3} = \frac{0 + (30-3) + (48-6)}{3} = 23ms$$



FCFS 방식 (2/2)

- **FCFS 평가**
  - 만약 처리 시간이 긴 프로세스가 CPU를 차지
    → 다른 프로세스들은 하염없이 기다려 시스템의
    효율성이 떨어짐
  - 만약 작업 중인 프로세스가 입출력 작업을 요청
    → CPU가 작업하지 않고 쉬는 시간이 많아져
    작업 효율이 떨어짐

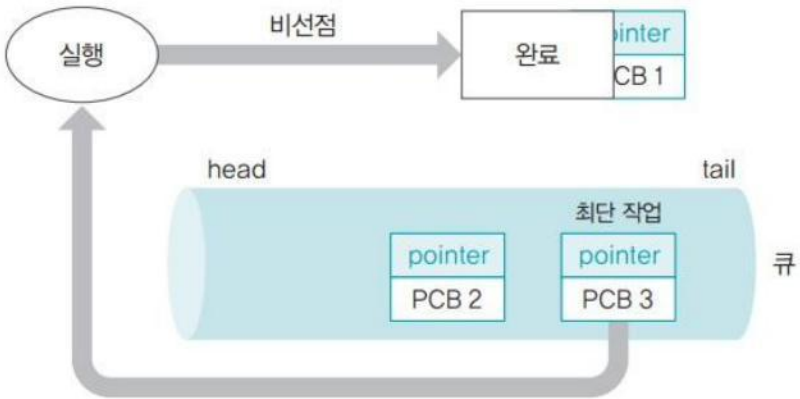


[콘보이 효과]

- **콘보이 효과**
  - 컨베이어 벨트에 작업물이 한 줄로 늘어서 있을 때,
    앞의 작업이 오래 걸려 뒤의 작업이 지연되는 현상

SJF 방식 (1/3)

- **SJF**
  - Shortest Job First : 최단 작업 우선
  - 준비 큐에 있는 프로세스 중에서 실행 시간이 가장 짧은 작업부터 CPU를 할당
  - **콘보이 효과를 완화**하여 시스템의 효율성을 높임

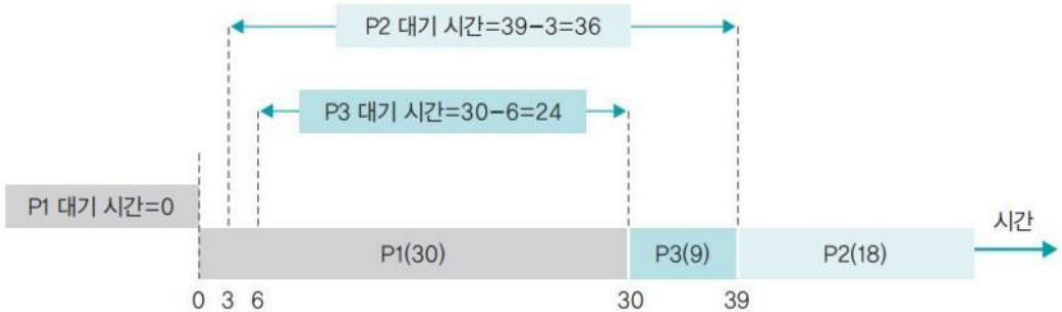


SJF 방식 (2/3)

- **SJF의 평균 대기시간 계산**

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

$$AVR_{(3)} = \frac{W_{(p1)} + W_{(p3)} + W_{(p2)}}{3} = \frac{0 + (30-6) + (39-3)}{3} = 20ms$$



※ P1는 가장 먼저 도착을 했으니, 대기하지 않고 먼저 처리
- P1처리 중 대기중인 다른 프로세스들에 대하여 우선순위를 변경

SJF 방식 (3/3)

- **SJF의 단점**
  - 운영체제가 프로세스의 종료 시간을 정확하게 예측하기 어려움
  - 작업 시간이 길다는 이유만으로 계속 뒤로 밀려 공평성이 현저히 떨어짐
    → **아사(starvation) 현상**

- **아사 해결 방법**
  - 에이징(Aging, 나이 먹기)
  - 순서를 양보할 때마다 나이를 한 살씩 먹임
  - 프로세스가 양보할 수 있는 상한선을 정함

HRN 방식 (1/2)

- **HRN**
  - Highest Response Ratio Next : 최고 응답률 우선
  - SJF 스케줄링에서 발생할 수 있는 **아사 현상을 해결**하기 위해 만들어진 비선점형 알고리즘
  - **대기 시간**과 **CPU 사용 시간**을 고려하여 스케줄링을 하는 방식
  - 여전히 **공평성이 위배**되어 많이 사용되지 않음
  - 프로세스의 우선순위를 결정하는 기준

$$\boldsymbol{Prority_i = \frac{Wait_i + CPU_i}{CPU_i}}$$

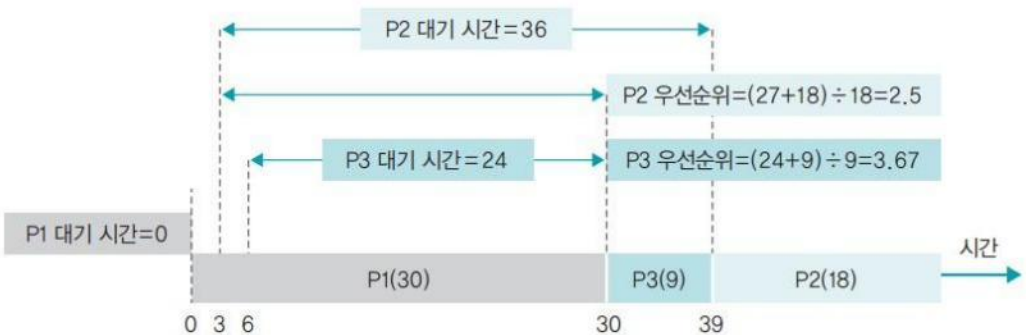
- $Prority_i$: i의 우선순위
- $Wait_i$: i의 대기 시간
- CPU_i : i의 CPU 사용시간

HRN 방식 (2/2)

- **HRN의 평균 대기시간 계산**
  ① 대기시간 0을 먼저 채택
  ② 우선순위(가중치)를 추출
  ③ 우선순위 순으로 프로세스를 나열(가중치가 높은 값이 먼저 채택)

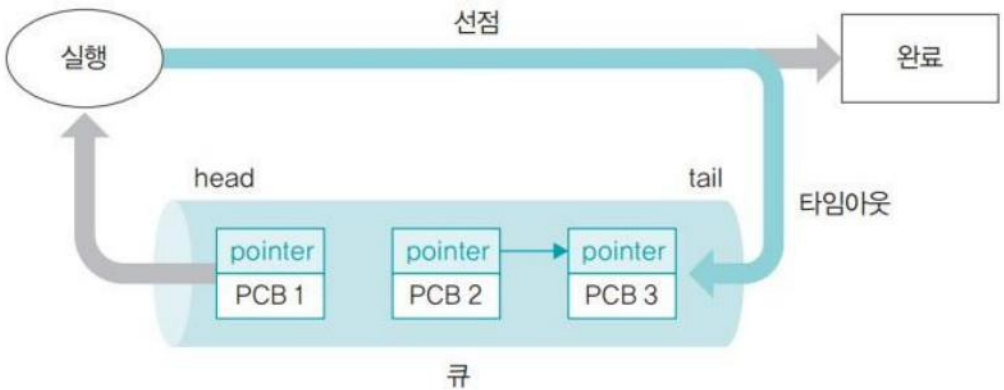
도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

$$AVR_{(3)} = \frac{W_{(p1)} + W_{(p3)} + W_{(p2)}}{3} = \frac{0 + (30-6) + (39-3)}{3} = 20ms$$



라운드 로빈 방식 (1/3)

- **라운드 로빈 스케줄링의 동작 방식 (1/2)**
  - 한 프로세스가 할당받은 시간(타임 슬라이스, 타임 슬롯) 동안 작업
  - 작업을 완료하지 못하면 준비 큐의 맨 뒤로 이동 전환
  - 다시 자기 차례를 기다림
  - **선점형 알고리즘** 중 가장 단순하고 대표적인 방식
  - 프로세스들이 작업을 완료할 때까지 계속 순환



라운드 로빈 방식 (2/3)

- **라운드 로빈 스케줄링의 동작 방식 (2/2)**
  - 효과적으로 작동하려면 문맥 교환에 따른 추가 시간을 고려하여 타임 슬라이스를 적절히 설정해야 함
  - 타임 슬라이스가 큰 경우
    - 하나의 작업이 끝난 뒤 다음 작업이 시작
    - FCFS 스케줄링과 다를 게 없음
  - 타임 슬라이스가 작은 경우
    - 문맥 교환이 너무 자주 일어남
    - 문맥 교환에 시간을 너무 많이 소모
      → 프로세스의 CPU 점유 시간을 단축(성능저하 발생)

라운드 로빈 방식 (3/3)

- **라운드 로빈 스케줄링의 성능**

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

※ 총 대기 시간
0(P1)+7(P2)+14(P3)+19(P1)+19(P2)+8(P1) = 67ms

※ 평균 대기 시간
67÷3 = 22.33ms

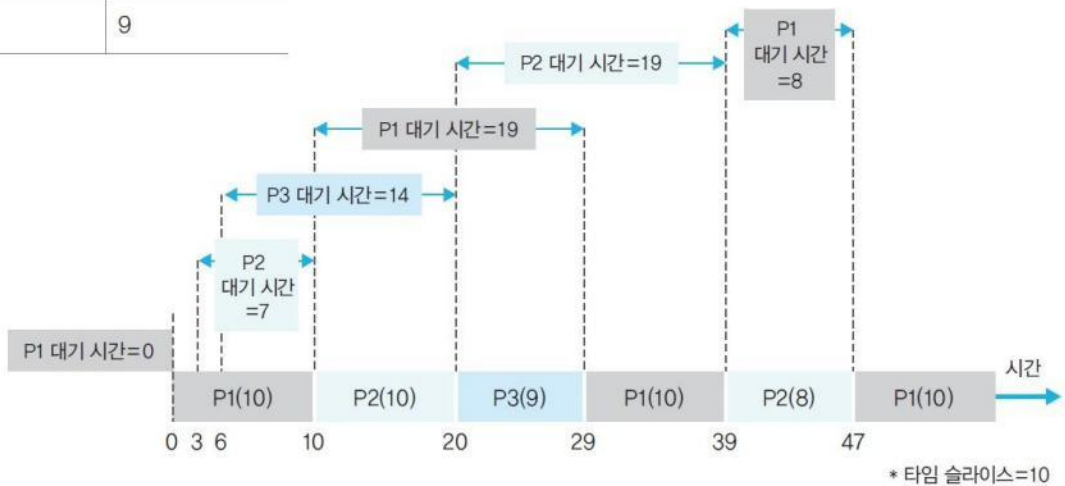
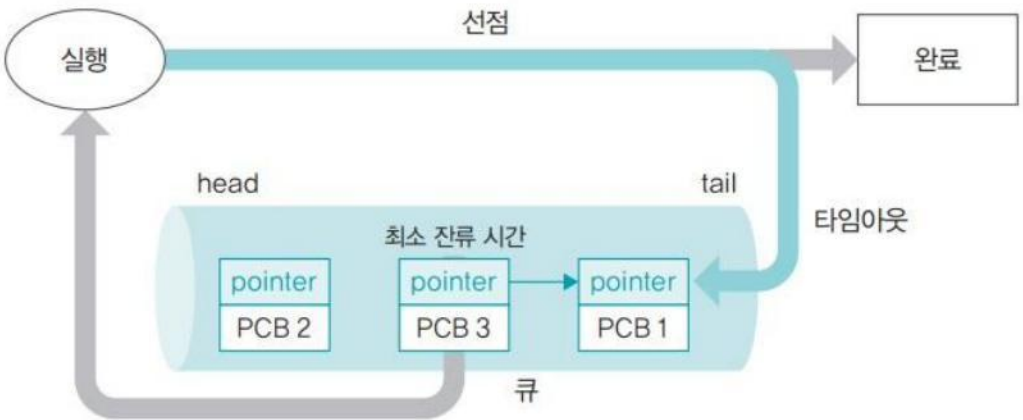


그림 4-21 라운드 로빈 스케줄링의 평균 대기 시간

SRT 방식 (1/2)

- **SRT**
  - 라운드 로빈 + SJF(Shortest Job First)
  - CPU의 할당은 남은 작업 시간이 가장 적은 프로세스를 선택
  - 현재 실행 중인 프로세스와 큐에 있는 프로세스의 남은 시간을 주기적으로 계산
  - 프로세스의 종료 시간을 예측하기 어렵고 **아사 현상** 발생 → 잘 사용하지 않음



SRT 방식 (2/2)

-

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

※ 총 대기 시간
0(P1)+4(P3)+16(P2)+27(P1) = 47ms

※ 평균 대기 시간
47÷3 = 15.66ms

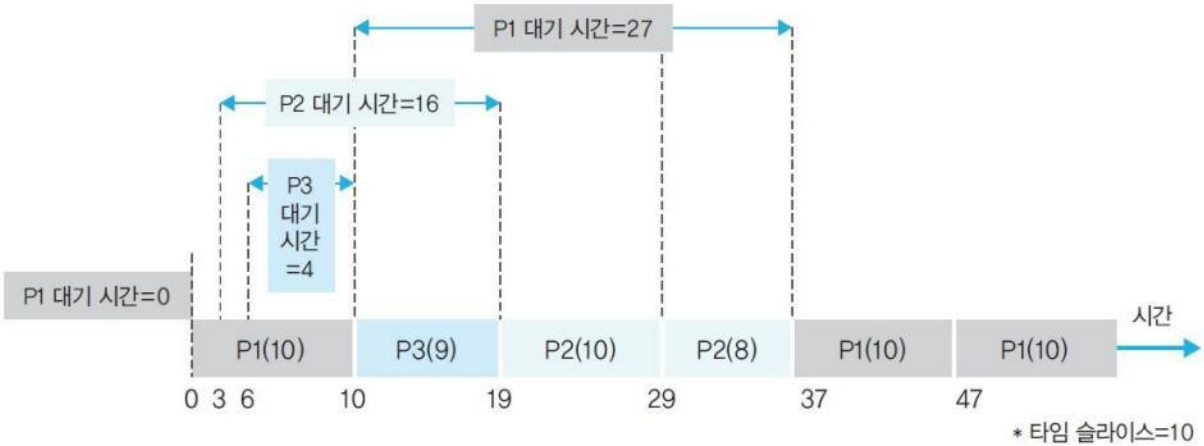
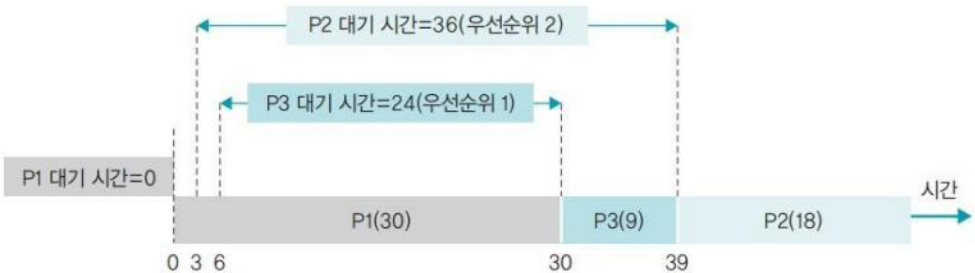


그림 4-23 SRT 스케줄링의 평균 대기 시간

기타 방식 (1/3)

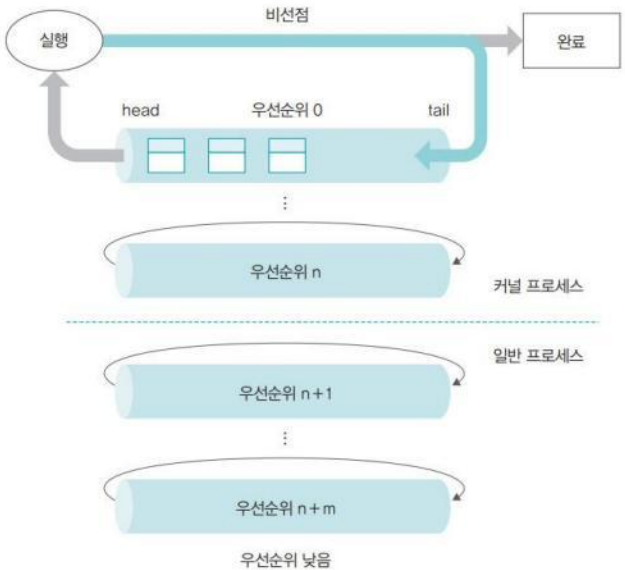
- **우선순위**
  - 프로세스의 중요도에 따른 우선순위를 반영한 스케줄링 알고리즘
  - 우선순위가 낮은 프로세스는 **아사 현상**이 발생될 수 있음
  - 비선점형 방식과 선점형 방식에 모두 적용할 수 있음(SJF, HRN, SRT)

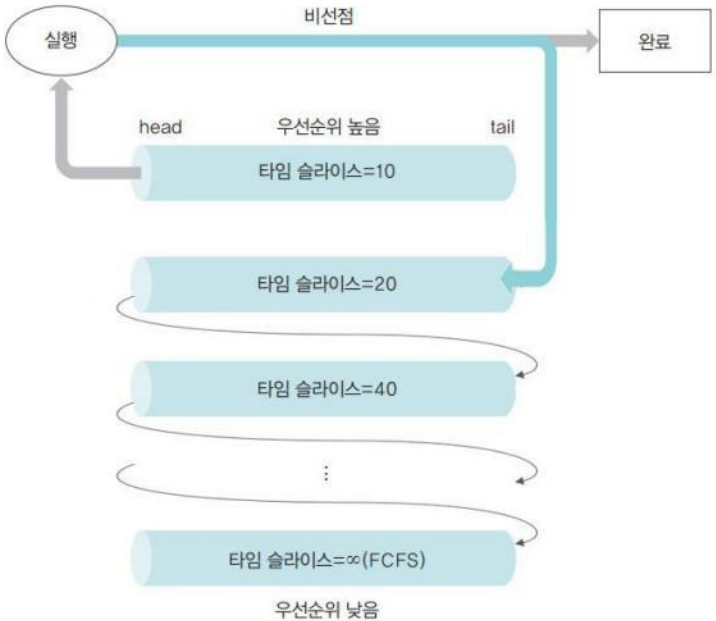
도착 순서	도착 시간	작업 시간	우선순위
P1	0	30	3
P2	3	18	2
P3	6	9	1



기타 방식 (2/3)

- **다단계 큐(MLQ; Multi-Level Queue)**
  - 우선순위에 따라 준비 큐를 여러 개 사용
    - 우선순위에 따라 해당 우선순위의 큐에 삽입(고정형 우선순위)
  - 상단의 큐의 모든 작업 완료 후 하단 큐 진행 → 아사 현상 유발



기타 방식 (3/3)

- **다단계 피드백 큐(MLFQ; Multi-Level Feedback Queue)**
  - 프로세스가 실행될 때마다 프로세스의 우선순위를 낮춤
  - 다단계 큐의 **낮은 우선순위 프로세스의 아사 현상을 해결**
  - 우선순위에 따라 타임 슬라이스의 크기가 다름
    - 우선순위가 낮아질수록 CPU를 얻을 확률이 낮아짐
    - 한번 CPU를 잡을 때 많이 작업하라는 기회를 줌
    - 마지막 큐의 프로세스는 무한대의 타임 슬라이스를 얻음 (FCFS 스케줄링 방식으로 동작)